

This homework has 8 questions, for a total of 100 points and 5 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

For bonus/extra-credit questions, we will provide very limited guidance in office hours and on Piazza, and we do not guarantee anything about the difficulty of these questions.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up*.

(0 pts) 0. **Before you start; before you submit.**

If applicable, state the name(s) and username(s) of your collaborator(s).

Solution: Shikha Nukala, desigirl

(10 pts) 1. **Self assessment.**

Carefully read and understand the posted solutions to the previous homework; you may also find the video “walkthroughs” in the Canvas Media Gallery helpful. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't turn in the previous homework, then (1) state that you didn't turn it in, and (2) pick a problem that you think is particularly challenging from the previous homework, and explain the answer in your own words. You may reference the answer key, but your answer should be in your own words.

Solution: 1) Did not turn in previous homework

2) Problem 5b

Consider the following language:

$$L_{\text{HALT} < 2^{|x|}} = \{(\langle M \rangle, x) : M(x) \text{ halts within } 2^{|x|} \text{ steps}\}.$$

Prove or disprove: $L_{\text{HALT} < 2^{|x|}}$ is decidable.

This is a decidable language because the language needs to halt within $2^{|x|}$ steps. Since the language takes in the input of x as a string, $|x|$ has to be a finite value. Thus, $2^{|x|}$ is a number of steps to which the algorithm runs finitely, and halts on any input because for finite input, we know that the language either accepts or rejects $(\langle M \rangle, x)$ based on whether the algorithm halts in $2^{|x|}$ steps or not. Therefore, for any input $(\langle M \rangle, x)$, the algorithm should be decidable within steps $2^{|x|}$.

2. Deciding undecidability.

For each of the following languages, show that it is undecidable via a Turing reduction from a language we have already shown is undecidable.

(As a refresher, you may wish to re-read Handout 2: Turing Reductions.)

- (10 pts) (a) $L_{\text{DISJOINT}} = \{(\langle M_1 \rangle, \langle M_2 \rangle) : M_1, M_2 \text{ are TMs and } L(M_1) \cap L(M_2) = \emptyset\}.$

Solution: We have already shown that the halting problem A_{TM} is undecidable as we cannot determine the the decision procedure works for all algorithms possible for any input. First, we can construct M_1 such that for an input $(\langle M \rangle, x)$, if M_1 halts on x , the TM accepts all inputs. Otherwise it rejects all inputs. We can construct M_2 such that for an input $(\langle M \rangle, x)$ the machine always rejects all inputs. Since we know that $L(M_1) \cap L(M_2) = \emptyset$, this means that they are disjoint sets. So for a specific input x , if M halts on x , then $L(M_1)$ is not empty but $L(M_2)$ is. Conversely, if it does not halt, then $L(M_1)$ is empty and the two sets are disjoint anyway. Since M_1 and M_2 represent the problem of A_{TM} , if this language were indeed decidable, then A_{TM} would also be decidable, which we know is not true. So L_{DISJOINT} cannot be decidable.

- (10 pts) (b) Let $S = \{\langle 183 \rangle, \langle 280 \rangle, \langle 281 \rangle, \langle 370 \rangle, \langle 376 \rangle\}$ and define the language

$$L_{\text{EECS}} = \{\langle M \rangle : M \text{ is a TM that accepts every string in } S \text{ and loops on every other string}\}.$$

Solution: Using the same undecidable known TM from the last part, we will give a turing reduction from L_{EECS} to A_{TM} . If we construct a TM called M, if the input is $(\langle M \rangle, x)$, then for an x such that x is in the set S , M will accept the input case. Otherwise, if x is not in the set S , if M simulates the current TM and halts on x , the algorithm will loop, otherwise M will accept. From this we can see that in both cases the TM always halts in both cases depending on whether x is in the set S or not. This assumes that it is possible to halt on every input, which we assume from a same TM input. If this were to be true, we would be able to determine A_{TM} as decidable, which is not true, so L_{EECS} must be undecidable as well.

3. **NP languages.** Prove that each of the following languages is in NP.

- (7 pts) (a) $\text{AFFORDABLEMENU} = \{\langle L, k \rangle : L \text{ is a list of menu items with positive integer prices, and there are at least } k \text{ distinct menu items that have total price exactly } 10k\}.$

Solution: First we begin by proving that L is a decidable language. We can do this by constructing a verifier V for L for $c = \text{some } k$.

V = On input $\langle L, k \rangle$:

- 1) non deterministically select at least k items
- 2) the total price of those items adding up to exactly $10k$.
- 3) if there is such a solution, accept, otherwise reject

$\langle L, k \rangle$ in L, if there is at least k items that have price exactly $10k$, D accepts

$\langle L, k \rangle$ not in L, if there are not k items that have price exactly $10k$, D rejects

From this we can see that the algorithm is verified, accepting or rejecting on any possible input k . Next we have to prove there is a nondeterministic polynomial time algorithm that runs with respect to the size of input k . From this algorithm, we can see that the algorithm runs with respect to the size of k menu items input and selecting those items out of the list, therefore the complexity can be worse case represented as $O(n^2)$ therefore this is a nondeterministic polynomial algorithm.

- (7 pts) (b) $\text{INDEPENDENTSET} = \{\langle G, k \rangle : G \text{ is an undirected graph with an independent set of size } k\}.$
An independent set in a graph is a subset C of the vertices for which the graph has no edge between any pair of vertices in C .

Solution: First we need to verify that INDEPENDENTSET has a verifier for $c = \text{some } k$. We can do this by constructing a verifier V for INDEPENDENTSET.

V = On input $\langle G, k \rangle$:

- 1) non deterministically select k vertices in G
- 2) check if those k vertices form an independent set in G to create subset C
- 3) if there is such a solution, accept, otherwise reject

$\langle G, k \rangle$ in INDEPENDENTSET, if there is a set with k vertices, D accepts

$\langle G, k \rangle$ not in INDEPENDENTSET, if there is not a set with k vertices, D rejects
From this we can see that the algorithm is verified, accepting or rejecting on any possible input k and graph G . Next we have to prove there is a nondeterministic polynomial time algorithm that runs with respect to the size of input k . From this algorithm, we can see that the algorithm runs with respect to the size of the graph and choosing k vertices, then iterating through those k vertices ensuring that the resulting subset is an independent set or not, therefore the complexity can be worse case represented as $O(n^2)$ therefore this is a nondeterministic polynomial algorithm.

4. Understanding **P**, **NP**, and poly-time mapping reductions.

- (7 pts) (a) We claim that $P \subseteq NP$, i.e., for any language $L \in P$, we have that $L \in NP$ as well. Here is an *incomplete* proof of this fact, which you will complete.

By the hypothesis that $L \in P$, there is an efficient Turing machine M that decides L . We define the following efficient verifier V for L .

- 1: **function** $V(x, c)$
- 2: **[MISSING PSEUDOCODE]**

State what the missing pseudocode should be, and prove that V is indeed an efficient verifier for L .

Solution: In the pseudocode, we need to show that function $V(x, c)$ returns the value of 1 (accepting) if there exists a c that completes that condition for some x in L . We also need to show that $V(x, c)$ returns the value 0 (rejecting) if x is not in L for any c . We also need to show that V takes polynomial time to run. Therefore we can see in the pseudocode

- 2) Run TM M for input x
- 3) if M accepts, ensure c is a valid certificate for x in L
- 4) return 1 if the conditions in 3 are satisfied
- 5) otherwise return 0 and reject in all other cases

This is an efficient verifier because firstly, we have proved that this indeed verifies L . We can see that M only accepts if both x is in L and c is a valid certificate for x , otherwise if either of those conditions are not met, M rejects. Additionally, this verifier runs in polynomial time since c is assumed to be of polynomial size, and the only comparison we perform in the verifier is whether c is valid for the input x .

- (10 pts) (b) Show that poly-time mapping reductions are transitive. That is, if $A \leq_p B$ and $B \leq_p C$, then $A \leq_p C$.

Solution: If we assume two functions that have polynomial time and are decidable, we can say that for function $a(x)$ the input x is in A if the function $a(x)$ is in B , similarly the input y is in B if the function $b(y)$ is in C . Therefore, to prove transitive property, we need to see that for some input z in A , the function $c(z)$ is in C for the condition $A \leq_p C$ to hold true. Therefore, we need to show that c is both decidable and runs in

polynomial time. We can rewrite c as $b(a(z))$ due to the transitive properties of the assumed statements above. Therefore since we assume b and a run in polynomial time, we can say that $c = b(a(z))$ also runs in polynomial time. For the second condition, we need so show that for some input z in A , the function $c(z)$ is in C . If we say z is in A , we can say that $a(z)$ is in B from the assumption above. Therefore, $b(a(z))$ is in C from the second assumption above. And vice versa, we can prove z is in A from $c(z)$ in C . Therefore c maps instances of A to C , so the condition $A \leq_p C$ holds true.

- (10 pts) (c) Let $C = A \cup B$ where both A, B are languages in NP. State, with proof, whether C is in NP for *all*, *some* (but not all), or *no* such A, B .

Solution: Since we know A and B are both languages in NP, we know that each language has a nondeterministic polynomial run time TM. We can call these machines M_a and M_b respectively. Therefore, if we have a C that is also an NP, there needs to be a machine M_c that is also a nondeterministic polynomial run time TM. We also know that if M_a OR M_b accepts on some input x , then M_c will also accept since $C = A \cup B$. We also know that M_c will non deterministically choose M_a OR M_b to run an input x on. Since M_c depends on M_a and M_b , we know that M_c must be a nondeterministic polynomial run time TM. We also know that A and B are verified, therefore M_c also verifies C . Since both NP conditions are fulfilled, C is also in NP for all such A and B languages.

Homework 7

5. Understanding coNP.

This question explores the complexity class $\text{coNP} = \{\bar{L} : L \in \text{NP}\}$, i.e., the set of languages whose complement languages are in NP. (Note that this is *not* the complement of NP itself, which is the set of languages that are not in NP.)

Recall that conceptually, NP is the set of languages whose “yes” instances can be verified efficiently, given suitable certificates. Symmetrically, coNP is the set of languages whose “no” instances can be verified efficiently (given suitable certificates).

- (5 pts) (a) Prove that P is closed under set complement. That is, for any $L \in \text{P}$, we have that $\bar{L} \in \text{P}$.

Solution: To prove P is closed under set complement, we already know that L is in P, so there exists a polynomial run time TM that decides L. We can call this TM M. Therefore, in order to prove $\bar{L}$ is also in P, there needs to exist a polynomial run time TM that decides $\bar{L}$. Therefore, we can consider writing a decider D for the TM M_1 for $\bar{L}$.

D = on input x:

- 1) simulate M on input x
- 2) if M accepts x then reject, otherwise accept

If we model M_1 after M, then the decisions the TM makes need to be the opposite of M as $\bar{L}$ is the complement. From M_1 , we can see that the machine runs in polynomial time because we assume M is polynomial which is only simulated in M_1 . To show that the TM is correct, we can see that if x is in L, then M accepts and M_1 rejects x which puts x in $\bar{L}$. We can also see if x is not in L, then M rejects and M_1 accepts x, which puts x in $\bar{L}$ so each input is decided. Therefore both conditions for P are achieved, and $\bar{L} \in \text{P}$, so P is closed under set complement.

- (9 pts) (b) Use the previous part to prove that if $\text{P} = \text{NP}$, the following set inclusions hold: (i) $\text{NP} \subseteq \text{coNP}$ (that is, for any $L \in \text{NP}$, we have $L \in \text{coNP}$), and (ii) $\text{coNP} \subseteq \text{NP}$.

Solution: i) We know here that if $\text{P} = \text{NP}$, P is closed under set complement from the last part. For $\text{NP} \subseteq \text{coNP}$, we know that coNP is the complement of the language passed into NP. Therefore for some language L in NP, there needs to exist a polynomial run time verifier $V(x,c)$ such that V is equal to 1 if x is in L and c is a valid certificate. Since $\text{P} = \text{NP}$, there is a TM M that is valid and decides in the same way as V for the language in NP. To show L is in coNP, $\bar{L}$ also has to be in NP. Using the previous problem, we know that P is closed under set complement, so $\bar{L}$ is also in NP if L is in NP. If this is true, then L is in coNP and the condition holds.

ii) If we have an L in coNP, then the complement $\bar{L}$ is in NP. Therefore there is a verifier that is polynomial run time that is $V(x,c) = 1$ if x is in $\bar{L}$ and c is a valid certificate for that input. Since $\text{P} = \text{NP}$, this verifier can be written in a TM M that decides the same decisions as the verifier. Now we need to show L is also in NP. So for L, there needs to exist a polynomial run time verifier such that $V_1(x,c)$ is 1 for x in L and certificate c that is valid. Therefore we can write the verifier based off of $\bar{L}$ verifier

$V_1(x,c)$

Homework 7

1) simulate M on input x and certificate c
 2) if M accepts, reject x , otherwise accept x
 from this verifier we see it runs in polynomial time as M is polynomial and it depends on M . We also can see that the verifier accepts x if and only if it is L , so it is verified. Therefore two conditions have been satisfied and L is in NP . Therefore the second condition holds.

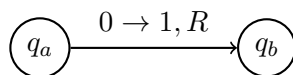
- (5 pts) (c) Conclude from the previous part that if $NP \neq coNP$, then $P \neq NP$.
 (Therefore, you could collect the \$1 million Clay Math prize “simply” by proving that $NP \neq coNP$!)

Solution: If we consider $NP \neq coNP$, we can see that from the previous part, if that is true, then either $coNP \subseteq NP$ or $NP \subseteq coNP$ are not true since both are only true if $NP = coNP$. If $NP \subseteq coNP$ is not true, then there is a language in NP that is not in $coNP$. That would mean since NP and $coNP$ are complements, there is a language in NP which complement is not in NP . This would cause the first case $P = NP$ to be false as it would not be closed under set complement. Similarly, if $coNP \subseteq NP$ is not true, then there is a language in $coNP$ that is not in NP . That would mean that since those two sets are complements, there is a language in $coNP$ which complement is not in $coNP$. This would cause the first case $P = NP$ to be false as it would not be closed under set complement because the complement set of NP is not closed under set complement. Both ways, P is not equal to NP if $NP \neq coNP$.

(10 pts) 6. **Cook-Levin windows.**

Recall from lecture that one critical ingredient in the proof of the Cook-Levin theorem is to specify which contents of the 2×3 “windows” of the verifier’s computation tableau are valid.

Specify *all* valid 2×3 window contents that could appear in two adjacent rows of the tableau, if those rows represent the verifier TM taking the following transition:



You do not need to provide justification.

For simplicity, you can assume that the symbols in the cells of the window are restricted to $\{0, 1, \perp, q_a, q_b\}$. Recall that there can be other symbols in the rows, such as the “boundary” symbol $\#$ and “separator” symbol $\$$, but we will ignore those here. To avoid explicitly writing out too many windows, you can use notation like $0/1/\perp$ to indicate that a cell could contain either 0, 1, or $\perp$. Also, to specify that a cell in the bottom row of a window must contain the same character as the cell above it, you can write “same”. For example, writing

0/1	$\perp$	1
same	0	$\perp$

represents the following two windows:

0	$\perp$	1
0	0	$\perp$

1	$\perp$	1
1	0	$\perp$

Be sure to include all valid window contents that represent the head of the TM being in *both* rows of the window, all those that represent the head being in *only one* of the rows of the window, and all those that represent the head not being in the window at all. Do not include any invalid windows.

Solution:

0	1	$\perp$
1	0	$\perp$

qa	0	$\perp$
1	qb	$\perp$

0	1	$\perp$
0	0	$\perp$

qa	0	$\perp$
1	qb	$\perp$

0	1	$\perp$
0	1	$\perp$

0	qa	0
0	1	qb

0	1	$\perp$
qa	1	$\perp$

qa	0	0
1	qb	0

0	1	$\perp$
0	1	$\perp$

0	1	$\perp$
same	same	same

0	0	$\perp$
same	same	same

0	1	$\perp$
same	same	same

1	0	$\perp$
same	same	same

			1	1	$\perp$
			same	same	same
			same	same	same
			same	same	same

(5 EC pts) 7. **Optional extra-credit problem: NP in exponential time.**

Let EXP be the class of all languages that are decidable in exponential time, i.e., in time $O(2^{n^k})$ for some constant k (where n is the length of the input).

It remains unknown whether $\text{NP} = \text{EXP}$, but it is known that $\text{P} \neq \text{EXP}$. Prove that $\text{NP} \subseteq \text{EXP}$. In other words, show that any problem that can be verified efficiently can be decided in exponential time.

Solution: Since we know that $\text{P} \neq \text{EXP}$, to prove $\text{NP} \subseteq \text{EXP}$ we need to prove all NP are in EXP even if we do not know if $\text{NP} = \text{EXP}$. Therefore any language in NP has to be decided in exponential time. If we say L is a language in NP, there has to be a polynomial run time verifier for L . So for $V(x,c) = 1$ there is some x in L and a certificate c that makes that condition true. Now if there can be a TM M such that L is decided in exponential time, we can see that any such language L with $V(x,c)$ in NP can be decided in exponential time. Therefore we can construct M

- 1) Certificates are generated such that they are all possible certificates of at most length 2^{n^k} with k being some constant and n being the length of the input.
- 2) Simulate V on x with each certificate c
- 3) If $V(x,c) = 1$ for any c in the generated lengths, accept x , if $V(x,c) = 0$ or not a valid certificate, reject

The complexity of the certificate generation is $O(2^{n^k})$ time and V is a polynomial run time verifier, so the complexity of M is exponential because of the length of x which means M is a decider in EXP. Therefore any language in NP can be decided in EXP, so all NP are in EXP, proving $\text{NP} \subseteq \text{EXP}$.